

Caso pedagógico 01 — Exposição de portas em produção

Contexto

Em ambientes de produção, decisões aparentemente “simples” de configuração podem introduzir riscos relevantes. Um exemplo clássico é **mapear portas de microsserviços diretamente no host** em arquivos de produção (`docker-compose.prod.yml`, `docker-compose.deploy.yml`). Embora isso facilite testes rápidos e inspeção manual, em produção esse padrão tende a **aumentar a superfície de ataque**, fragmentar controles de segurança e tornar a operação mais cara e difícil de padronizar.

Este caso apresenta o problema, os impactos, o racional arquitetural e o “delta” aplicado nos arquivos de *compose* para alinhar a solução a boas práticas modernas de segurança e operação.

Problema identificado

Os microsserviços da aplicação estavam com blocos `ports:` em arquivos de produção, expondo serviços de aplicação diretamente no host. Isso faz com que requisições externas consigam atingir serviços específicos **sem passar por um ponto único de entrada** (gateway/ingress/reverse proxy).

Por que isso é um problema em produção?

Quando cada serviço publica sua própria porta no host: - A aplicação deixa de ter um **perímetro controlado** (um único ponto de entrada). - Controles como **TLS, autenticação, rate limiting e WAF** precisam ser replicados em múltiplos serviços. - *Observability* (logs, métricas, tracing) tende a ficar **despadronizada** ou incompleta. - A governança operacional fica mais difícil (regras, certificados, auditoria, mudanças).

Impacto potencial

A exposição direta de portas em produção pode levar a:

1. **Acesso direto a serviços sem passar pelo gateway**
2. Requisições externas podem contornar o roteamento e as políticas centralizadas.
3. **Exploração de endpoints internos**
4. Rotas que foram desenhadas para tráfego interno (ex.: endpoints técnicos, rotas administrativas, health detalhado, endpoints de debug) ficam mais facilmente alcançáveis.
5. **Perda de padronização de segurança**

6. TLS e regras de segurança distribuídos por serviço aumentam risco de inconsistência e erro de configuração.

7. Observabilidade “furada”

8. Logs, métricas e tracing podem deixar de passar pelo mesmo pipeline, reduzindo capacidade de diagnóstico e auditoria.

9. Aumento de custo de manutenção

10. Mais pontos para atualizar certificados, regras de firewall, rate limiting, autenticação e políticas.

Solução fundamentada

Em produção, a recomendação é **expor publicamente apenas o gateway/ingress**.

- **Serviços de aplicação (microserviços)** devem se comunicar **pela rede interna** do ambiente (no caso do Docker Compose, por uma *network* interna, como `aurora_network`).
- Para necessidades internas de verificação/saúde, prefira `expose:` em vez de `ports:`.
- `expose:` **documenta e torna disponível internamente** a porta do container na rede do Compose.
- `ports:` **mapeia para o host**, tornando o serviço acessível externamente (o que deve ser evitado em produção para microserviços).

Princípios e boas práticas associados

- **Defense-in-depth:** múltiplas camadas de proteção; reduzir exposição direta é uma camada essencial.
- **Single entry point:** um único ponto público facilita controle, auditoria e manutenção.
- **Hardening centralizado:** TLS, WAF, rate limiting e autenticação concentrados no gateway.
- **12-factor (port binding com disciplina):** serviços escutam suas portas internamente, mas a exposição pública é controlada por um componente dedicado.

Implementação (delta aplicado)

A mudança não altera a lógica dos serviços — apenas a forma como são expostos.

1) `docker-compose.prod.yml`

- Removidos todos os blocos `ports:` de:
 - `users-service`
 - `auth-service`
 - `events-service`
- Os serviços passam a escutar **somente na rede interna** `aurora_network`.

2) `docker-compose.deploy.yml`

- Removidos os blocos `ports:` de:

- `users-service`
- `auth-service`
- `events-service`
- `registrations-service`

• A comunicação entre serviços permanece **interna**.

Observação: em **Dev/QA**, manter `ports:` em `docker-compose.dev.yml` é aceitável e útil para testes locais e depuração.

Efeito prático (o que muda no comportamento do sistema)

Após o ajuste: - Tráfego externo **não alcança diretamente** os microsserviços. - O acesso público passa a exigir um **gateway/reverse proxy** para roteamento. - Isso **reduz exposição**, força passagem por **autenticação e observabilidade centralizadas** e facilita aplicar políticas consistentes.

Checklist didático para validação em aula

Use este checklist para os alunos verificarem se entenderam e aplicaram corretamente:

1. Em produção, **apenas o gateway** possui `ports:` (ex.: `80:80`, `443:443`).
 2. Microsserviços **não** possuem `ports:` em `docker-compose.prod.yml` e `docker-compose.deploy.yml`.
 3. Microsserviços (se necessário) usam `expose:` para documentação/uso interno.
 4. Todos os serviços estão na mesma *network* interna (ex.: `aurora_network`).
 5. A comunicação ocorre via **DNS interno do Docker** (nome do serviço + porta).
 6. Em Dev/QA, `ports:` pode existir para testes, mas não deve vazar para produção.
-

Perguntas para discussão (para consolidar aprendizado)

1. Qual é a diferença prática entre `expose:` e `ports:`?
 2. Se um microsserviço precisa ser acessível publicamente, por que isso deveria ocorrer via gateway e não por uma porta no host?
 3. Quais controles (TLS, WAF, rate limiting, auth) ficam mais simples quando existe um único ponto de entrada?
 4. Em caso de incidente, por que reduzir a superfície de ataque ajuda na resposta e na mitigação?
-

Conclusão

Expor portas diretamente no host para cada microsserviço em produção costuma ser um atalho que aumenta risco e custo. Ao mover a exposição pública para um **gateway/ingress** e manter os microsserviços apenas na rede interna, a arquitetura ganha em: - segurança, - consistência operacional, - observabilidade, - e facilidade de manutenção.

Esse padrão é um dos pilares para operar microsserviços de forma profissional e escalável em ambientes reais.